TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA
04

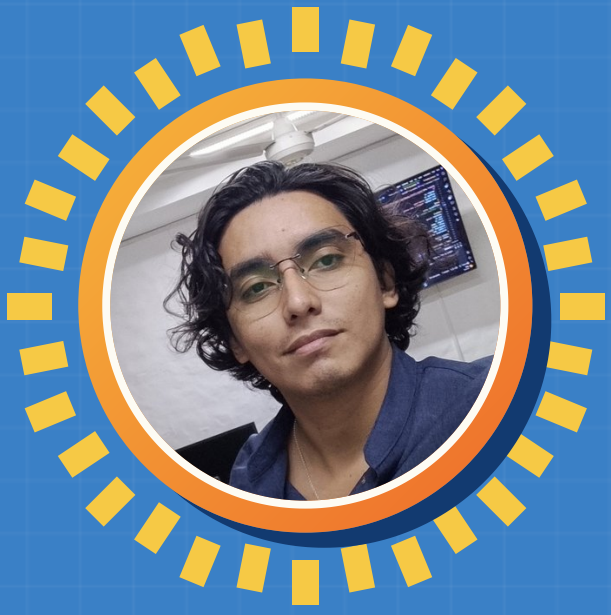
// Unidad I · Fundamentos de Java

ABSTRACCIÓN, POLIMORFISMO E INTERFACES

PROGRAMAR CONTRA UN CONTRATO

EN ESTA LECTURA

01 El problema: un if por cada tipo
02 Polimorfismo a fondo
03 Abstracción
04 Interfaces
05 ¿Clase abstracta o interfaz?
06 Del contrato a Spring Boot
07 Caso integrador
08 Errores comunes
09 Para practicar



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología Francisco "El Chele" Moreno

Fundamentos de Desarrollo Backend con Java
Noviembre 2026

CAPÍTULO 01

1 EL PROBLEMA: UN IF POR CADA TIPO

> En la Lectura 03 conociste los cuatro pilares de un vistazo. Esta lectura se detiene en los dos últimos, **abstracción** y **polimorfismo**, y en su herramienta principal, las **interfaces**, porque Spring Boot se apoya en ellas todo el tiempo.

Imagina una empresa de envíos. Cada tipo de envío cobra distinto. La primera idea suele ser un método que pregunta el tipo y decide:

Polimorfismo.java · CalculadoraConIf

```
class CalculadoraConIf {

    public double calcularCosto(String tipo, double pesoKg) {
        if (tipo.equals("estandar")) {
            return 50 + pesoKg * 10;
        } else if (tipo.equals("moto")) {
            return 30 + pesoKg * 8;
        } else if (tipo.equals("camion")) {
            return 50 + pesoKg * 10 + 200;
        }
        return 0;       // si se nos olvidó un tipo, nadie avisa: cobra 0
    }
}
```

Funciona, pero tiene tres problemas que crecen junto con el programa:

- **Cada tipo nuevo obliga a abrir este método.** Y a abrir todos los que tengan un `if` parecido.
- **Los errores no avisan.** Si alguien escribe `"dron"`, el programa compila, corre y cobra `C$0.0`.
- **Una clase sabe demasiado.** Conoce las reglas de todos los tipos, y no cada tipo las suyas.

Tres herramientas, tres preguntas

Herramienta	La pregunta que responde	Lo que logras
Polimorfismo	¿Quién decide qué código se ejecuta?	El objeto real, no un `if`.
Abstracción	¿Qué necesito saber para usar esto?	Solo el *qué*; el *cómo* queda escondido.
Interfaces	¿Qué debe saber hacer esta pieza?	Un contrato: piezas que se pueden cambiar.

El código de esta lectura

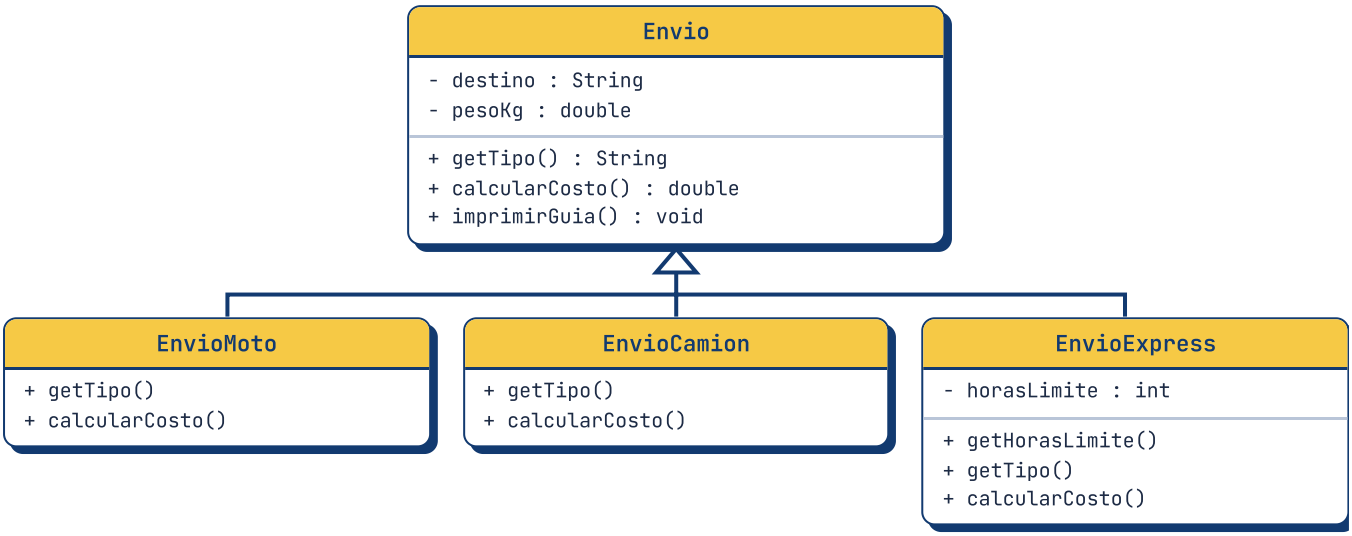
Los ejemplos están en el repositorio del curso. Ábrelos junto a esta lectura y ejecútalos:

Archivo	Qué muestra
`Polimorfismo.java`	Envíos: ligadura dinámica, `instanceof` y la trampa de la sobrecarga.
`Abstraccion.java`	Correo y reportes: esconder detalles y clases abstractas.
`Interfaces.java`	Capacidades, `default`, `static` e inyección de dependencias.
`CasoIntegrador.java`	Matrícula de cursos: todo junto en un mismo sistema.

CAPÍTULO 02 · PILAR 3

2 POLIMORFISMO A FONDO

Polimorfismo significa “muchas formas”: **un mismo mensaje** produce respuestas distintas según el objeto que lo recibe. Quien llama ya no pregunta el tipo; cada objeto sabe responder.



Polimorfismo.java · la superclase

```java
class Envio {

    private String destino;
    private double pesoKg;

    // ... constructor y getters

    public String getTipo() {
        return "Estándar";
    }

    public double calcularCosto() {
        return 50 + pesoKg * 10;
    }

    // Este método está escrito UNA sola vez, pero getTipo() y calcularCosto()
    // ejecutan la versión del objeto real: el padre llama al código del hijo.
    public void imprimirGuia() {
        System.out.println(getTipo() + " a " + destino + ": C$" + calcularCosto());
    }
}
```

Una subclase tiene dos formas de sobrescribir un método: **reemplazarlo** por completo o **reutilizarlo** con `super` y agregar lo suyo.

Polimorfismo.java · dos subclases

```
class EnvioMoto extends Envio {

    // ... constructor y getTipo()

    @Override
    public double calcularCosto() {
        return 30 + getPesoKg() * 8;     // reemplaza por completo la regla del padre
    }
}

class EnvioCamion extends Envio {

    // ... constructor y getTipo()

    @Override
    public double calcularCosto() {
        return super.calcularCosto() + 200;      // reutiliza la regla del padre y agrega
    }
}
```

Dos decisiones, dos momentos

Esta es la idea central del capítulo. En una línea como `Envio envio = new EnvioMoto(...)`, Java toma dos decisiones en momentos distintos:

Quién decide	Cuándo	Se fija en	Qué decide
El compilador	Al compilar	El tipo de la **variable**	Qué métodos puedes llamar.
La JVM	Al ejecutar	El **objeto** real	Qué versión se ejecuta.

A la segunda decisión se le llama **ligadura dinámica**. Por eso una misma variable puede dar resultados distintos:

Polimorfismo.java · dentro de main()

```
Envio envio = new EnvioMoto("Masaya", 3);
System.out.println("Costo: C$" + envio.calcularCosto());    // versión de EnvioMoto

// La misma variable puede apuntar luego a otro objeto de la familia
envio = new EnvioCamion("Estelí", 100);
System.out.println("Costo: C$" + envio.calcularCosto());    // versión de EnvioCamion
```

Salida

```
Costo: C$54.0
Costo: C$1250.0
```

Código que no pregunta el tipo

El polimorfismo se aprovecha de verdad cuando un método recibe el tipo del padre. La clase `Despacho` solo conoce a `Envio`:

Polimorfismo.java · Despacho

```
public void imprimirGuias(Envio[] envios) {
    for (int i = 0; i < envios.length; i++) {
        envios[i].imprimirGuia();
    }
}

public double calcularTotal(Envio[] envios) {
    double total = 0;
    for (int i = 0; i < envios.length; i++) {
        total += envios[i].calcularCosto();     // cada objeto responde a su manera
    }
    return total;
}
```

Polimorfismo.java · dentro de main()

```
Envio[] envios = {
    new Envio("León", 2),
    new EnvioMoto("Masaya", 3),
    new EnvioCamion("Estelí", 100),
    new EnvioExpress("Granada", 2, 4)
};

Despacho despacho = new Despacho();
despacho.imprimirGuias(envios);
System.out.println("Total del día: C$" + despacho.calcularTotal(envios));
```

Salida

```
Estándar a León: C$70.0
Moto a Masaya: C$54.0
Camión a Estelí: C$1250.0
Express a Granada: C$140.0
Total del día: C$1514.0
```

COMPARA CON EL IF

Para agregar un `EnvioDron` basta con **crear la clase** y ponerla en el arreglo. `Despacho` no cambia ni una línea. Con la versión del `if`, habría que abrir `CalculadoraConIf` y cualquier otro método que preguntara el tipo.

Cuando necesitas algo propio del hijo: instanceof

El método `getHorasLimite()` solo existe en `EnvioExpress`. Con una variable de tipo `Envio`, el compilador no te deja llamarlo, aunque el objeto real lo tenga. Para eso existe `instanceof`, que revisa el objeto real:

Polimorfismo.java · dentro de main()

```
Envio urgente = new EnvioExpress("Rivas", 1, 6);

// urgente.getHorasLimite();   // ERROR: la variable es de tipo Envio y
//                             // Envio no tiene ese método.

// instanceof revisa el objeto real. Si coincide, "express" ya viene
// convertida al tipo EnvioExpress y se puede usar dentro del if.
if (urgente instanceof EnvioExpress express) {
    System.out.println("Debe llegar en " + express.getHorasLimite() + " horas.");
}
```

NO ABUSES DE INSTANCEOF

Una cadena de `if (x instanceof A) ... else if (x instanceof B) ...` es el mismo problema del capítulo 1 con otra ropa. Antes de usarla, pregúntate si ese comportamiento debería ser un método sobrescrito.

La trampa: la sobrecarga la decide el compilador

La **sobrescritura** se decide al ejecutar, pero la **sobrecarga** se decide al compilar, mirando solo el tipo de la variable. El mismo objeto puede entrar por métodos distintos:

Polimorfismo.java · Despacho tiene dos versiones de describir()

```
EnvioExpress comoExpress = new EnvioExpress("Jinotega", 1, 8);
Envio comoEnvio = comoExpress;      // el MISMO objeto, otra variable

// Sobrecarga: se elige al COMPILAR, según el tipo de la variable.
despacho.describir(comoExpress);    // versión (EnvioExpress)
despacho.describir(comoEnvio);      // versión (Envio), aunque el objeto es express

// Sobrescritura: se elige al EJECUTAR, según el objeto real.
System.out.println("Tipo real: " + comoEnvio.getTipo());    // Express
```

Salida

```
describir(EnvioExpress): urgente, 8 horas
describir(Envio): un envío a Jinotega
Tipo real: Express
```

Reglas para sobrescribir

Regla	Si no la cumples
Mismo nombre y mismos parámetros.	Creas un método nuevo. `@Override` te avisa del error.
No puedes reducir la visibilidad (de `public` a nada).	Error de compilación.
`private`, `static` y los constructores no se sobrescriben.	No hay ligadura dinámica para ellos.
Un método `final` no se puede sobrescribir.	Error de compilación. Lo verás en el capítulo 3.

CAPÍTULO 03 · PILAR 4

3 ABSTRACCIÓN

> Abstraer es mostrar **qué hace** algo y esconder **cómo lo hace**. En Java lo verás en dos niveles: un método público simple que esconde pasos privados, y las clases abstractas.

Nivel 1: un método simple, detalles escondidos

Quien usa `ClienteCorreo` solo necesita conocer `enviar()`. No sabe, ni le importa, que por dentro hay cuatro pasos:

Abstraccion.java · ClienteCorreo

```
class ClienteCorreo {

    public void enviar(String destinatario, String mensaje) {
        conectar();
        autenticar();
        transmitir(destinatario, mensaje);
        desconectar();
    }

    private void conectar() {
        System.out.println("  (abre la conexión con el servidor)");
    }

    // ... autenticar(), transmitir() y desconectar() también son private
}
```

Si mañana cambia la forma de conectarse, se modifica solo `conectar()`, y ningún programa que use `enviar()` se entera. La palabra `abstract` no aparece, y aun así esto ya es abstracción.

	Encapsulamiento	Abstracción
Se ocupa de	Proteger los **datos**.	Simplificar el **uso**.
Pregunta	¿Quién puede cambiar esto?	¿Qué necesita saber quien lo usa?
Ejemplo	`saldo` es `private`.	`enviar()` esconde cuatro pasos.

Nivel 2: la clase abstracta como plantilla

¿Qué sería “un reporte” a secas, sin ser de ventas ni de inventario? Nada concreto. Cuando un concepto solo existe a través de sus variantes, lo correcto es declararlo `abstract`:

Abstraccion.java · la plantilla

```
abstract class Reporte {

    private String titulo;

    public Reporte(String titulo) {     // lo llaman las hijas con super(...)
        this.titulo = titulo;
    }

    // Método abstracto: sin cuerpo y termina en punto y coma.
    // Toda hija concreta está OBLIGADA a implementarlo.
    protected abstract void imprimirDetalle();

    // Método normal: la secuencia está escrita aquí una sola vez y las hijas
    // solo aportan la parte que cambia.
    // final impide que una hija lo sobrescriba y altere la estructura.
    public final void imprimir() {
        System.out.println("---------- " + titulo + " ----------");
        imprimirDetalle();              // se ejecuta la versión del objeto real
        System.out.println();
    }
}
```

Abstraccion.java · una subclase

```
class ReporteAsistencia extends Reporte {

    private int presentes;
    private int matriculados;

    public ReporteAsistencia(String titulo, int presentes, int matriculados) {
        super(titulo);
        this.presentes = presentes;
        this.matriculados = matriculados;
    }

    @Override
    protected void imprimirDetalle() {
        double porcentaje = (double) presentes / matriculados * 100;
        System.out.println("Presentes : " + presentes + " de " + matriculados);
        System.out.println("Asistencia: " + porcentaje + " %");
    }
}
```

Abstraccion.java · dentro de main()

```
Reporte[] reportes = {
    new ReporteVentas("Cafetería", 18, 6540.50),
    new ReporteInventario("Bodega", 120, 7),
    new ReporteAsistencia("Clase 04", 21, 25)
};

for (int i = 0; i < reportes.length; i++) {
    reportes[i].imprimir();
}
```

```
Salida
---------- Cafetería ----------
Ventas: 18
Total : C$6540.5

---------- Bodega ----------
Unidades en bodega: 120
Con stock bajo    : 7

---------- Clase 04 ----------
Presentes : 21 de 25
Asistencia: 84.0 %
```

UN PATRÓN CON NOMBRE

Un método del padre que fija los pasos y deja algunos a las hijas se llama **método plantilla** (*Template Method*). El padre no conoce a sus hijas, pero llama a su código: abstracción y polimorfismo trabajando juntos.

Lo que el compilador te exige

Si intentas…	El compilador responde
Crear un objeto: `new Reporte("x")`	`Reporte is abstract; cannot be instantiated`
Una hija que no implementa el método abstracto	`… is not abstract and does not override abstract method imprimirDetalle() in Reporte`
Un método abstracto con llaves `{ }`	`abstract methods cannot have a body`

Estos errores son una ayuda: es imposible olvidar una parte del contrato y descubrirlo cuando el programa ya está corriendo. Una hija solo se libra de implementar el método si ella también se declara `abstract`.

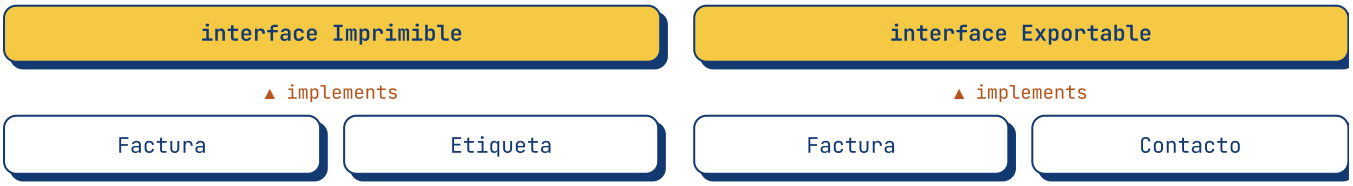
CAPÍTULO 04

4 INTERFACES

> Una interfaz es un **contrato**: dice qué debe saber hacer una clase, sin decir cómo. La clase que la firma con `implements` queda obligada a cumplirlo.

“Es un” contra “puede”

La herencia expresa identidad: un `EnvioMoto` **es un** `Envio`. Una interfaz expresa una **capacidad**, y esa capacidad la pueden tener clases que no tienen nada que ver entre sí. Una factura y una etiqueta no son parientes, pero las dos **pueden** imprimirse:



Una clase solo puede heredar de **una** clase, pero puede implementar **varias** interfaces. `Factura` firma las dos:

Interfaces.java · contrato y clases

```
interface Imprimible {
    void imprimir();        // public y abstract por defecto
}

// Una clase solo puede extender a UNA clase, pero puede implementar VARIAS interfaces.
class Factura implements Imprimible, Exportable {

    private int numero;
    private double total;

    // ... constructor

    @Override
    public void imprimir() {
        System.out.println("Factura #" + numero + " por C$" + total);
    }

    @Override
    public String exportar() {
        return Exportable.unir(String.valueOf(numero), String.valueOf(total));
    }
}

class Etiqueta implements Imprimible {

    private String texto;

    // ... constructor

    @Override
    public void imprimir() {
        System.out.println("[ " + texto + " ]");
    }
}
```

Una interfaz también sirve como tipo, así que el polimorfismo funciona igual que con una superclase:

Interfaces.java · dentro de main()

```
Imprimible[] cola = {
    new Factura(1001, 1350),
    new Etiqueta("Frágil")
};

for (int i = 0; i < cola.length; i++) {
    cola[i].imprimir();
}
```

Salida

```
Factura #1001 por C$1350.0
[ Frágil ]
```

SIEMPRE PUBLIC

Los métodos de una interfaz son `public` aunque no lo escribas. Por eso, al implementarlos, **debes** escribir `public`. Si lo olvidas, el compilador responde con un error que empieza con: `imprimir() in Factura cannot implement imprimir() in Imprimible`.

Lo que una interfaz puede contener

Interfaces.java · Exportable

```
interface Exportable {

    // Atributo de interfaz: siempre es constante (public static final)
    String SEPARADOR = ";";

    String exportar();

    // default: método con cuerpo. Quien implementa lo hereda o lo sobrescribe.
    default void mostrarExportacion() {
        System.out.println("Exportando → " + exportar());
    }

    // static: pertenece a la interfaz, no a los objetos
    static String unir(String a, String b) {
        return a + SEPARADOR + b;
    }
}
```

Elemento	¿La clase debe escribirlo?	Para qué sirve
Método abstracto	**Sí**, es obligatorio.	El contrato en sí.
Constante	No.	Valores fijos del contrato. No guarda estado.
Método `default`	No, es opcional.	Comportamiento común, o agregar métodos sin romper las clases que ya existen.
Método `static`	No.	Utilidades: `Exportable.unir("uno", "dos")`.

CAPÍTULO 05

5 ¿CLASE ABSTRACTA O INTERFAZ?

> No compiten: resuelven problemas distintos y muchas veces aparecen juntas. La Lectura 03 las comparó en una tabla; aquí tienes cinco preguntas para decidir.

Pregunta	Respuesta	Usa
¿Tus clases comparten **atributos o código**?	Sí	Clase abstracta
¿Puedes decir “X **es un** Y” sin dudar?	Sí	Clase abstracta
¿Clases sin parentesco comparten una **capacidad**?	Sí	Interfaz
¿Quieres poder **cambiar la implementación** sin tocar a quien la usa?	Sí	Interfaz
¿Una clase necesita **varios** de estos comportamientos?	Sí	Interfaces: solo hay una superclase

LAS DOS A LA VEZ

`class CursoVirtual extends Curso implements Certificable` se lee como una frase: **es un** curso y **puede** emitir certificado. En Java, `extends` siempre se escribe antes que `implements`. Lo verás funcionando en el capítulo 7.

CAPÍTULO 06

6 DEL CONTRATO A SPRING BOOT

> Aquí está la razón por la que este tema importa tanto en backend. Un sistema se organiza en **capas**, y cada capa habla con la siguiente a través de un contrato.



El repositorio: un contrato para guardar datos

Un **repositorio** es la pieza que guarda y busca datos. El contrato dice qué se puede hacer, no dónde se guarda. Puede haber varias implementaciones:

Interfaces.java · el contrato y la primera implementación

```java
record Libro(String isbn, String titulo) { }

interface LibroRepositorio {
    void guardar(Libro libro);
    Libro buscarPorIsbn(String isbn);   // devuelve null si no lo encuentra
    int contar();
}

// Implementación 1: guarda en un arreglo. Se pierde al cerrar el programa.
class RepositorioEnMemoria implements LibroRepositorio {

    private Libro[] libros = new Libro[100];
    private int cantidad = 0;

    @Override
    public void guardar(Libro libro) {
        if (cantidad < libros.length) {
            libros[cantidad] = libro;
            cantidad++;
        }
    }

    // ... buscarPorIsbn() y contar()
}
```

Interfaces.java · la segunda implementación

```java
// Implementación 2: no guarda nada y siempre responde lo mismo.
// Sirve para probar el servicio sin depender de una base de datos.
class RepositorioDePrueba implements LibroRepositorio {

    // ... guardar() y contar()

    @Override
    public Libro buscarPorIsbn(String isbn) {
        return new Libro(isbn, "Libro de prueba");
    }
}
```

Inyección de dependencias

El servicio necesita un repositorio, pero **no lo crea**: lo recibe por el constructor. A esto se le llama **inyección de dependencias**. El servicio no contiene ni un `new RepositorioEnMemoria()`, así que no sabe cuál le tocó:

Interfaces.java · BibliotecaServicio

```java
class BibliotecaServicio {

    private final LibroRepositorio repositorio;

    public BibliotecaServicio(LibroRepositorio repositorio) {
        this.repositorio = repositorio;
    }

    public boolean registrar(String isbn, String titulo) {
        if (repositorio.buscarPorIsbn(isbn) ≠ null) {
            System.out.println("  [X] Ya existe un libro con ISBN " + isbn + ".");
            return false;
        }
        repositorio.guardar(new Libro(isbn, titulo));
        System.out.println("  [OK] Registrado: " + titulo);
        return true;
    }

    // ... mostrar()
}
```

Interfaces.java · dentro de main()

```java
LibroRepositorio enMemoria = new RepositorioEnMemoria();
BibliotecaServicio biblioteca = new BibliotecaServicio(enMemoria);

biblioteca.registrar("978-1", "Don Quijote");
biblioteca.registrar("978-2", "Azul...");
biblioteca.registrar("978-1", "Otro con el mismo ISBN");    // rechazado
biblioteca.mostrar("978-2");
biblioteca.mostrar("000-0");                                // no existe

BibliotecaServicio dePrueba = new BibliotecaServicio(new RepositorioDePrueba());
dePrueba.mostrar("cualquier-isbn");
```

```
Salida
  [OK] Registrado: Don Quijote
  [OK] Registrado: Azul...
  [X] Ya existe un libro con ISBN 978-1.
  Libro[isbn=978-2, titulo=Azul...]
  [X] No hay ningún libro con ISBN 000-0.
  Libro[isbn=cualquier-isbn, titulo=Libro de prueba]
```

El mismo `BibliotecaServicio` funcionó con dos repositorios distintos sin cambiar una línea. Cuando llegue MySQL, solo hará falta una tercera implementación.

Lo que vas a reconocer en Spring Boot

Hoy conectaste las piezas a mano. Spring hace ese trabajo por ti, pero la idea es la misma. Este fragmento es **solo para mirar**: todavía no compila, porque necesita un proyecto de Spring.

```
Vista previa · Spring Boot
public interface LibroRepositorio extends JpaRepository<Libro, Long> { }

@Service
public class BibliotecaServicio {

    private final LibroRepositorio repositorio;

    public BibliotecaServicio(LibroRepositorio repositorio) {
        this.repositorio = repositorio;
    }
}
```

Hoy, a mano	En Spring Boot
Tú escribes `RepositorioEnMemoria`.	Declaras la interfaz y Spring genera la implementación.
Tú escribes `new BibliotecaServicio(...)` en `main`.	Spring crea el objeto porque lleva `@Service`.
Tú pasas el repositorio al constructor.	Spring lo pasa por ti, usando ese mismo constructor.

NO TE ASUSTES

`<Libro, Long>` se llama **genéricos**: indica con qué tipos trabaja el repositorio, en este caso libros cuyo identificador es un `Long`. Lo verás con calma cuando lleguemos a Spring.

CAPÍTULO 07

CASO INTEGRADOR

> Un sistema de **matrícula de cursos** reúne todo lo anterior. El código completo está en `CasoIntegrador.java`; aquí va lo esencial.

Pieza	Concepto	Por qué
`Curso`	Clase abstracta	Todos tienen nombre, horas y cupos, pero no existe un curso “genérico”.
`calcularPrecio()`	Polimorfismo	Presencial, virtual y charla calculan distinto.
`Certificable`	Interfaz: capacidad	Algunos cursos dan certificado y las charlas no.
`Notificador`	Interfaz: contrato	Avisar por correo o por SMS sin tocar el servicio.
`ServicioMatricula`	Inyección de dependencias	Recibe su `Notificador` por el constructor.

CasoIntegrador.java · un curso

```java
// extends e implements juntos: ES UN Curso y PUEDE emitir certificado
class CursoPresencial extends Curso implements Certificable {

    public CursoPresencial(String nombre, int horas, int cupos) {
        super(nombre, horas, cupos);
    }

    @Override
    public double calcularPrecio() {
        return getHoras() * 40 + 150;   // C$150 de materiales
    }

    @Override
    public String getModalidad() {
        return "Presencial";
    }

    @Override
    public String emitirCertificado(String estudiante) {
        return "Certificado de " + getHoras() + " horas para " + estudiante;
    }
}
```

CasoIntegrador.java · el servicio

```java
class ServicioMatricula {

    private final Notificador notificador;
    private double recaudado;

    public ServicioMatricula(Notificador notificador) {
        this.notificador = notificador;
    }

    public boolean matricular(String estudiante, String contacto, Curso curso) {
        if (!curso.reservarCupo()) {
            notificador.enviar(contacto, curso.getNombre() + " ya no tiene cupos.");
            return false;
        }

        recaudado += curso.calcularPrecio();    // no pregunta qué tipo de curso es
        notificador.enviar(contacto, "Matrícula confirmada en " + curso.getNombre() + ".");

        // No todos los cursos tienen esta capacidad: aquí sí se pregunta
        if (curso instanceof Certificable certificable) {
            System.out.println("  " + certificable.emitirCertificado(estudiante));
        }
        return true;
    }

    // ... getRecaudado()
}
```

CasoIntegrador.java · dentro de main()

```java
Curso[] catalogo = {
    new CursoPresencial("Backend con Java", 30, 2),
    new CursoVirtual("Git y GitHub", 20, 50),
    new Charla("Tu primer empleo en tecnología", 2, 1)
};

ServicioMatricula porCorreo = new ServicioMatricula(new NotificadorEmail());

porCorreo.matricular("Ana", "ana@correo.com", catalogo[0]);
porCorreo.matricular("Luis", "luis@correo.com", catalogo[2]);
porCorreo.matricular("Rosa", "rosa@correo.com", catalogo[2]);   // sin cupo

ServicioMatricula porSms = new ServicioMatricula(new NotificadorSms());

porSms.matricular("Carla", "8888-1234", catalogo[1]);
```

Salida

```
  Email a ana@correo.com: Matrícula confirmada en Backend con Java.
  Certificado de 30 horas para Ana
  Email a luis@correo.com: Matrícula confirmada en Tu primer empleo en tecnología.
  Email a rosa@correo.com: Tu primer empleo en tecnología ya no tiene cupos.
  SMS al 8888-1234: Matrícula confirmada en Git y GitHub.
  Constancia digital para Carla
```

¿Y ESTE INSTANCEOF?

En el capítulo 2 te pedimos no abusar de `instanceof`, y aquí se usa uno. Es razonable, porque no pregunta “¿qué tipo de curso eres?” para calcular algo, sino “¿tienes esta capacidad?”. Si mañana aparece otro curso certificable, este código no cambia.

8 CAPÍTULO 08
ERRORES COMUNES

Error	Qué hacer
El compilador dice `cannot find symbol` al llamar a un método del hijo.	La variable es del tipo del padre. Declara el método en el padre o usa `instanceof`.
`method does not override or implement a method from a supertype`	`@Override` encontró un error de escritura, como `calcularcosto()`. Corrige el nombre o los parámetros.
Una clase abstracta con un solo método y sin atributos.	No comparte nada: probablemente debería ser una interfaz.
Una interfaz para cada clase, aunque solo vaya a existir una versión.	Un contrato sirve cuando hay, o puede haber, varias implementaciones. Si no, agrega ruido.
Interfaces gigantes con muchos métodos.	Divídelas por capacidad. Mejor `Imprimible` y `Exportable` que una sola que obliga a todo.
El servicio hace `new RepositorioEnMemoria()` por dentro.	Queda atado a esa clase. Recibe el contrato por el constructor.

9 CAPÍTULO 09
PARA PRACTICAR

> Resuelve estos ejercicios en tu carpeta `clase-04`. Crea un archivo `.java` por clase, como en un proyecto real.

01 **Un tipo más** básico

Agrega `EnvioDron` a la jerarquía de `Polimorfismo.java`: cuesta `C$100 + C$20 por kg`. Ponlo en el arreglo y comprueba que `Despacho` no necesitó ningún cambio.

02 **Otro reporte** básico

Agrega `ReporteCalificaciones` a `Reporte`. Recibe el total de estudiantes y los aprobados, e imprime el porcentaje de aprobación.

03 **Instrumentos** intermedio

Crea la clase abstracta `Instrumento` con `nombre` y el método abstracto `tocar()`. Crea `Marimba` y `Guitarra`, guárdalas en un arreglo y haz que todas suenen con un solo ciclo.

04 **Una capacidad nueva** intermedio

Crea la interfaz `Descontable` con `double aplicarDescuento(double total)`. Haz que dos clases sin parentesco, `Cupon` y `TarjetaCliente`, la implementen con reglas distintas.

05 **Un canal más** intermedio

En `CasoIntegrador.java`, agrega `NotificadorWhatsApp` y úsalo en un `ServicioMatricula`. ¿Tuviste que cambiar `ServicioMatricula`? Explica por qué.

06 **Repositorio de estudiantes** `reto`

Crea el `record Estudiante(carnet, nombre)` y la interfaz `EstudianteRepositorio`, con una implementación en memoria y otra de prueba. Escribe un `MatriculaServicio` que las reciba por el constructor y rechace carnets repetidos.

Resumen del documento

Concepto	Palabras clave	Recuerda
Polimorfismo	`@Override` `super` `instanceof`	La variable decide qué puedes llamar; el objeto, qué se ejecuta.
Sobrecarga	mismo nombre, otros parámetros	Se decide al compilar, por el tipo de la variable.
Abstracción	`private` `abstract` `final`	Muestra el qué y esconde el cómo.
Interfaz	`interface` `implements` `default`	Un contrato. Una clase puede firmar varios.
Inyección	constructor + interfaz	No crees lo que necesitas: recíbelo. Así trabaja Spring.

Antes de la próxima clase

- Haber ejecutado los cuatro archivos de esta lectura y comparado su salida.
- Poder explicar por qué `describir(comoEnvio)` no entra por la versión de `EnvioExpress`.
- Poder decir, para un caso dado, si conviene una clase abstracta o una interfaz.
- Con esto cierras la Unidad I. Lo siguiente: **Git y GitHub**, antes de entrar a Spring Boot.

Referencias

Oracle. *The Java™ Tutorials: Polymorphism.* docs.oracle.com/javase/tutorial/java/landl/polymorphism.html

Oracle. *The Java™ Tutorials: Abstract Methods and Classes.* docs.oracle.com/javase/tutorial/java/landl/abstract.html

Oracle. *The Java™ Tutorials: Interfaces.* docs.oracle.com/javase/tutorial/java/landl/createinterface.html

OpenJDK. *JEP 394: Pattern Matching for instanceof.* openjdk.org/jeps/394

Spring. *Dependency Injection.* docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026